

CSE 422L: Artificial Intelligence Lab

Training hotel booking dataset

Syeda Mahmuda, 22301142, Yusrat Sadia Nailat, 22311003

Abstract—This project investigates the performance of several machine learning models for predicting hotel booking cancellations. Four models K-Nearest Neighbors (KNN), Logistic Regression, Naive Bayes, and a Neural Network were implemented and evaluated using metrics such as accuracy, precision, recall, F1-score, and Area Under the Curve (AUC). The results show that each model has different strengths and weaknesses. Logistic Regression performs well in identifying non-cancellations but misses many actual cancellations, while Naive Bayes achieves high recall but produces a large number of false positives. KNN provides a balanced performance but still makes moderate errors. Among all models, the Neural Network achieves the best overall performance, showing a good balance between correctly identifying both classes and minimizing errors. This study highlights the importance of selecting an appropriate model based on the specific requirements of the problem, particularly the trade-off between detecting cancellations and avoiding false predictions.

Index Terms—unsupervised and supervised machine learning algorithm, KNN, logistic regression, neural networks, naive bayes

I. INTRODUCTION

IN the hospitality industry, predicting booking cancellations is an important task that can help hotels manage resources, reduce revenue loss, and improve customer service. With the increasing availability of data, machine learning techniques provide effective tools for identifying patterns and making accurate predictions about customer behavior. This study focuses on applying and comparing different machine learning models to predict whether a booking will be canceled. The models considered include K-Nearest Neighbors (KNN), Logistic Regression, Naive Bayes, and a Neural Network. Each of these models has different assumptions and learning mechanisms, which lead to differences in performance. To evaluate the models, standard classification metrics such as accuracy, precision, and Area Under the Curve (AUC) are used. In addition, confusion matrix analysis is performed to better understand the types of errors made by each model. The goal of this work is not only to identify the best-performing model but also to understand the trade-offs between different approaches. This helps in selecting a suitable model depending on whether the priority is to minimize false predictions or to detect as many cancellations as possible.

II. METHODOLOGY

In section, we describe the methodology for implement our project. But before that, we describe our dataset. This dataset contains hotel reservation records. There are total of 119,390 rows (number of bookings) and 32 features where each row illustrates each booking. There is no single target variable in this dataset. Depending on our problem statement, we

can categorize the problem as a classification or regression problem. Say, if the target variable is *is_canceled*, then the problem is a *classification problem*. Since the output is discrete (binary categories) rather than continuous, the task is binary classification. If the target variable is chosen to be *the length of stay*, then the problem could be a regression problem. There are both quantitative and categorical features in this dataset.

The list of quantitative features is

- 1) is_canceled (binary variable)
- 2) lead_time
- 3) arrival_date_year
- 4) arrival_date_week_number
- 5) arrival_date_day_of_month
- 6) stays_in_weekend_nights
- 7) stays_in_week_nights
- 8) adults
- 9) children
- 10) babies
- 11) is_repeated_guest (binary variable)
- 12) previous_cancellations
- 13) previous_bookings_not_canceled
- 14) booking_changes
- 15) agent
- 16) company
- 17) days_in_waiting_list
- 18) adr (Average Daily Rate)
- 19) required_car_parking_spaces
- 20) total_of_special_requests

The list of categorical feature is

- 1) hotel (Resort Hotel / City Hotel)
- 2) arrival_date_month
- 3) meal
- 4) country
- 5) market_segment
- 6) distribution_channel
- 7) reserved_room_type
- 8) assigned_room_type
- 9) deposit_type
- 10) customer_type
- 11) reservation_status
- 12) reservation_status_date

It is important to note that the categorical features need to be encoded numerically since machine learning models cannot interpret strings.

A correlation matrix shows how strongly different variables are linearly related to each other. Correlation matrix is essentially a block matrix where each matrix element can range

from -1.0 to 1.0 . For simplicity, we assume that our target variable is *is_canceled*.

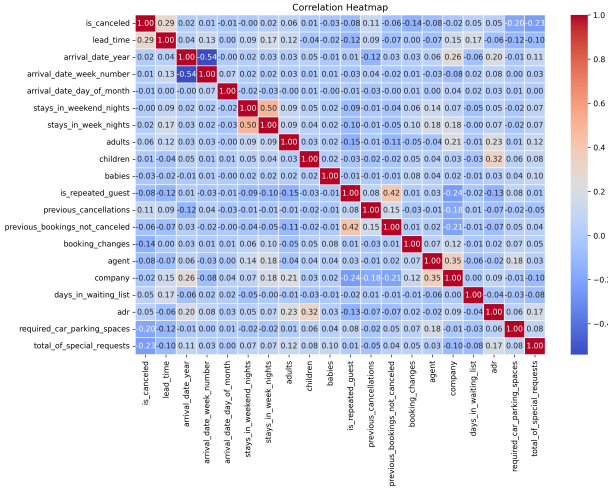


Fig. 1. Correlation of all input and output features.

We see that this variable has a correlation value 0.29 with *lead_time*, implying that bookings made earlier are more likely to be canceled. On the other hand, *is_canceled* has a negative correlation value of -0.23 with total of special request, suggesting that the cancellation of booking is not strongly influenced by the number of special requests made. The correlation value between *arrival_date_year* and *arrival_date_week_number* is -0.54 , indicating a moderate negative relationship. This should ideally suggest that as the year increases, the recorded week number tends to decrease. However, this relationship arises from the structure of the calendar rather than any meaningful underlying pattern, and therefore may not be useful for predictive modeling.

III. CORRELATION ANALYSIS

To understand the relationship between different features and booking cancellation, we computed Pearson correlation coefficients. This method capture different types of relationships between variables and provide a comprehensive understanding of feature importance.

A. Pearson Correlation

Pearson correlation measures the strength of a **linear relationship** between two variables. It is defined as:

$$\rho_{X,Y} = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{\sum_{i=1}^n (x_i - \mu_X)(y_i - \mu_Y)}{\sqrt{\sum_{i=1}^n (x_i - \mu_X)^2} \sqrt{\sum_{i=1}^n (y_i - \mu_Y)^2}}$$

where μ_X and μ_Y are the means of X and Y , and σ_X , σ_Y are their standard deviations.

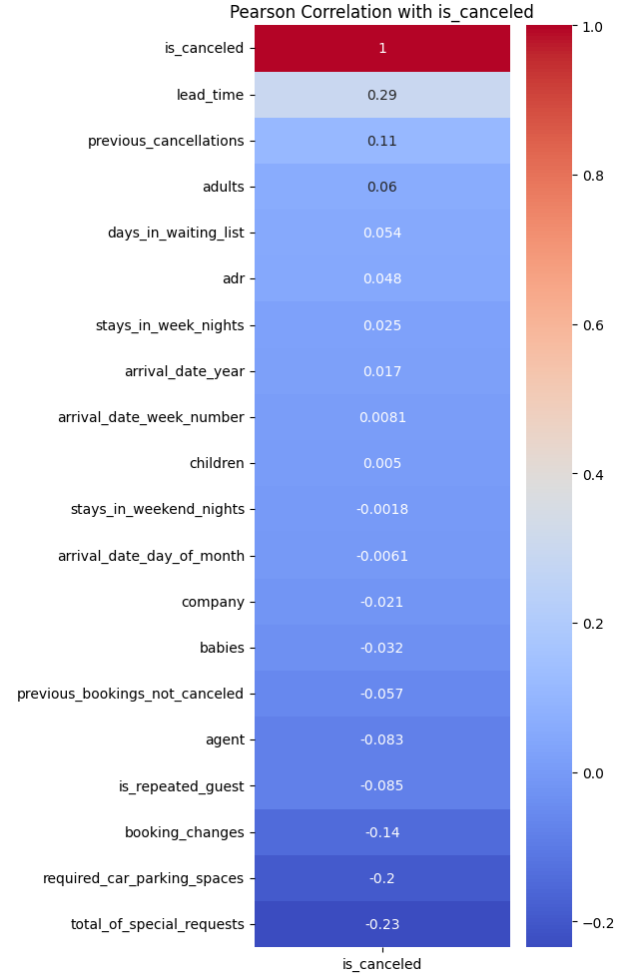


Fig. 2. different statistical correlations with *is_canceled*

B. Interpretation of Results

From the correlation analysis, several important insights can be observed:

- **Lead time** shows the strongest positive correlation with cancellation (approximately 0.29–0.32), indicating that bookings made far in advance are more likely to be canceled.
- **Previous cancellations** also exhibit a positive correlation, suggesting that customers with a history of cancellations are more likely to cancel again.
- **Total special requests** shows a strong negative correlation (approximately -0.23 to -0.26), indicating that customers who make more requests are less likely to cancel, reflecting higher engagement.
- **Required car parking spaces** and **booking changes** also have negative correlations, implying that more committed customers are less likely to cancel.
- Most other features have weak correlations ($|\rho| < 0.1$), indicating little to no linear relationship with the target variable.

In this part of the project, we will check whether the dataset is balanced or imbalanced. For this particular problem, we assume that our target variable is *is_canceled*. To examine

whether the dataset is imbalanced, we analyze the distribution of the target variable *is_canceled*. The bar chart shows that the number of non-canceled bookings (class 0) is significantly higher than the number of canceled bookings (class 1). Therefore, the dataset is imbalanced, as the classes do not have an equal number of data points.

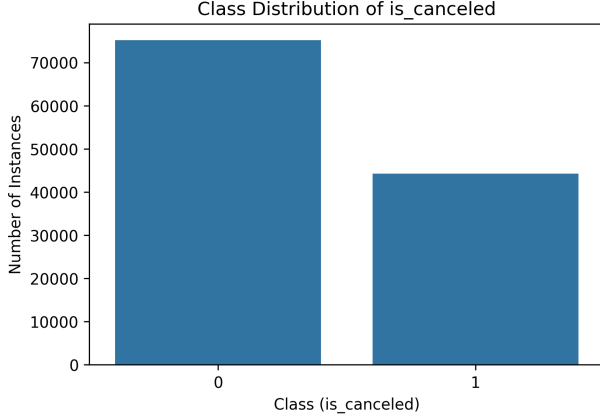


Fig. 3. class distribution for target variable *is_canceled*

IV. PROBLEM STATEMENT I

Our first problem is to develop a predictive model that determines whether a hotel booking will be canceled or not, based on the available features in the dataset. Given input variables such as *lead_time*, *adr*, *customer_type*, and other booking-related attributes, the task is to predict the target variable *is_canceled*. Since the output is binary (0 for not canceled and 1 for canceled), this is formulated as a binary classification problem.

We start off by dropping columns that have many null values. We first drop *company* column. We identify that there are many missing values in column *agent*, *country*, and *children*. We replace the missing values with the mode for column *agent*, "unknown" for *country*, and 0 for *children*. We also drop two more columns *reservation_status* and *reservation_status_date* since they cause data leakage, meaning that these columns contain post-booking information that directly reveals the outcome which makes the model unrealistically accurate. For this particular dataset, we do not delete any row for pre processing purpose. For machine learning purpose, we encode the categorical data into numerical forms. We used one-hot encoding for features such as hotel, meal, deposit type, customer type, market segment, country, and arrival date month because these categorical variables do not possess a natural ordering among their categories. On the other hand, ordinal encoding is used for features that contain an inherent hierarchy or natural ordering. Ordinal encoding assigns numerical values to categories while preserving their relative order.

For simplicity of analysis, we group *lead_time* by different time frame. After categorizing *lead_time* values, we compare cancellation rate across different groups.

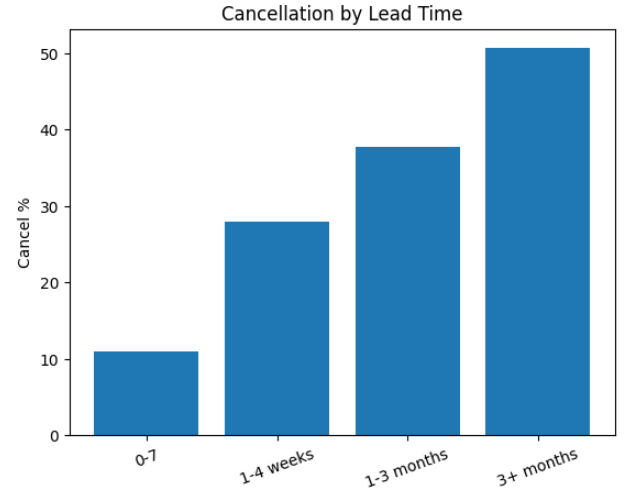


Fig. 4. Cancellation rate across different lead time categories

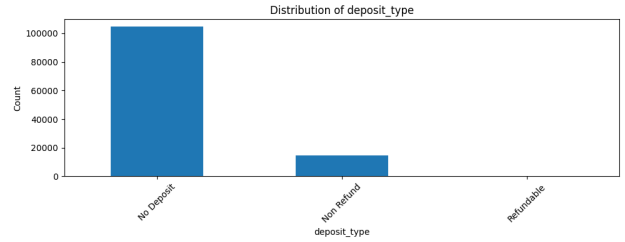


Fig. 5. type of deposit vs count plot

Finally, we categorize our dataset based on the type of customers. We see that number of transient customer is the highest.

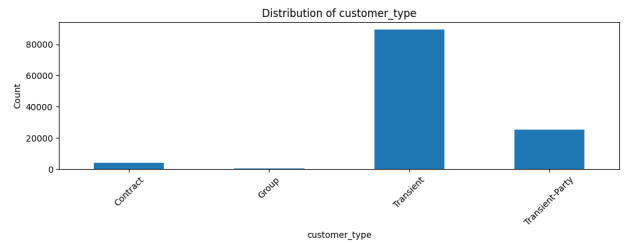


Fig. 6. number of bookings vs. market segment. Most bookings come from online travel agents.

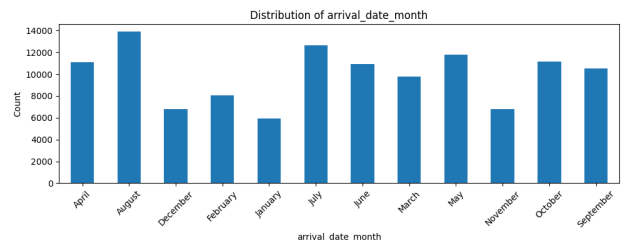


Fig. 7. number of bookings in different months of the year.

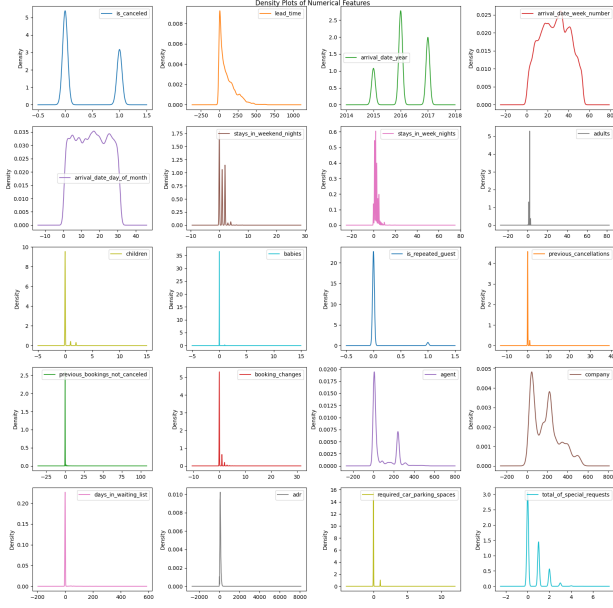


Fig. 8. Kernel density estimation plot for estimation for each features. KDE captures the probability density for each feature. KDE provides a smooth approximation of the underlying data distribution, where higher peaks indicate regions with more frequent observations, while wider distributions represent greater variability in feature values. The density plots show that most features are skewed and concentrated near small values, indicating sparse behavior and the need for scaling and transformation.

V. MODEL TRAINING AND TESTING

In this part, we split our data into training set and test set in order to evaluate the model on unseen data and prevent overfitting. The large portion of data belongs to the training set is which roughly 80% of the total data points.

A. Unsupervised learning

We treat first this problem as an unsupervised learning problem, meaning that we treat this dataset as an unlabeled dataset to find patterns and structure. We are basically exploring the dataset to find hidden insights. One of such unsupervised learning algorithm is K -means. K -means clustering is an unsupervised machine learning algorithm that groups data into K clusters by assigning each point to the nearest cluster center (centroid) and updating the centroids until they stabilize. Here, we group similar bookings into clustering based on two features – *lead_time*, and *children*. The feature choice implies that similarity between bookings is defined based on booking timing and family size. To determine the number of appropriate clustering, we employ elbow method. At first, we calculate the mean distance of a member of the cluster and the centroid to measure how close each data point is to its cluster centroid. We calculate SSE [Sum of Squared Errors (also called inertia)]. We use variable number of clusters (K value) and generate the SSE value for each iteration. Therefore, they are plotted and used to find the optimum value of K . Therefore, inertia is basically the measure of how tightly the data points are grouped into clusters. Here, we see that there is an elbow at $k = 3$ when we consider our two choice of features to be lead time and number of children. Therefore, the optimal number

of clustering for this dataset is 3. We also compute K -means for two other features – *adr* and *lead_time*. We see that inertial drops slowly from $k = 1$ to $k = 9$ with no sharp band. This is because *adr* is continuous and *shew* and does not form any group.

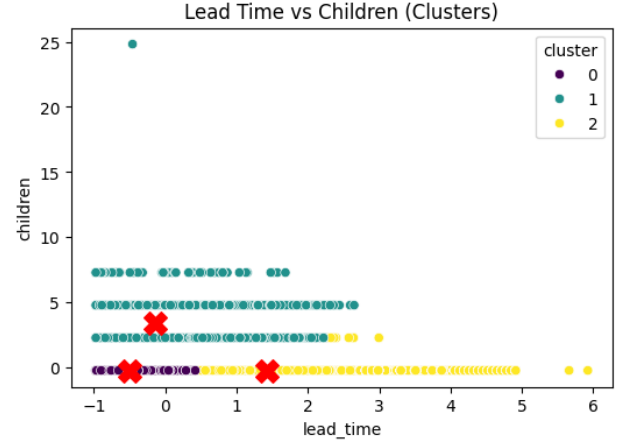


Fig. 9. K -means clustering for lead time and number of children

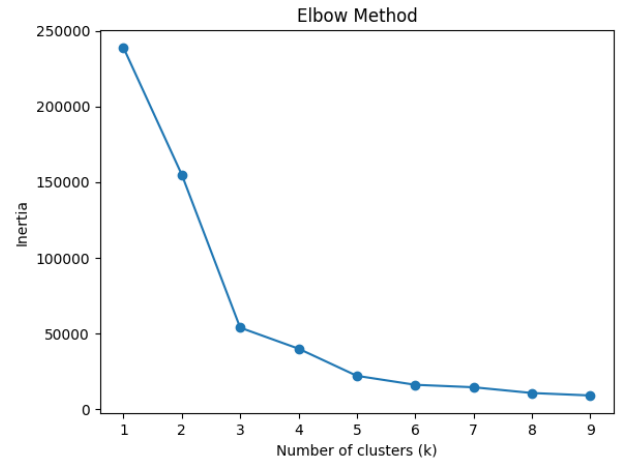


Fig. 10. Elbow curve for optimal cluster selection for lead time and children (aka family size)

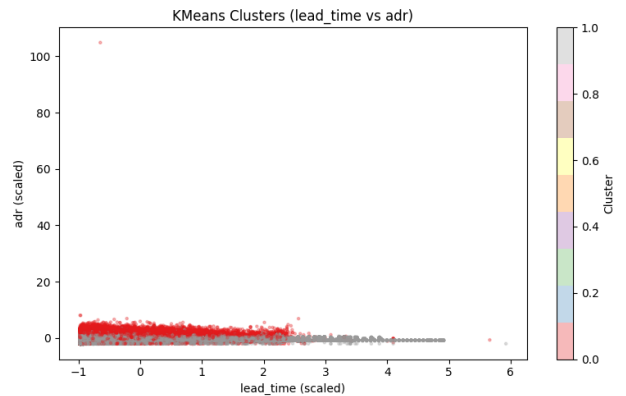


Fig. 11. K -means clustering for lead time and *adr*

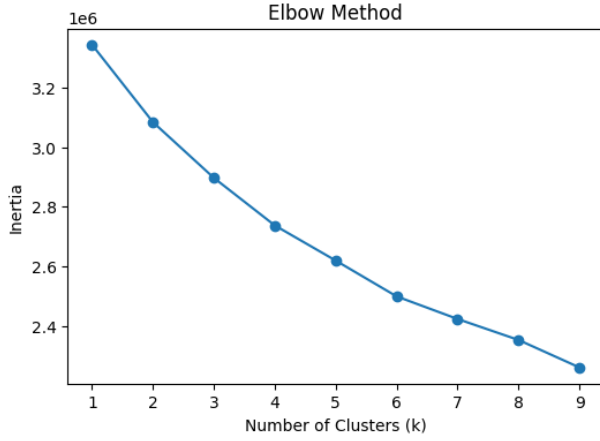


Fig. 12. Elbow curve for optimal cluster selection for lead time and adr

B. Supervised learning

In this section, we treat our problem as an supervised learning problem, meaning that we treat the hotel booking dataset as a labeled dataset. We will use different supervised learning models to predict whether a booking will be canceled.

1) *K-Nearest Neighbors (KNN)*: *K*-Nearest Neighbors (KNN) is a supervised, non-parametric learning algorithm used for classification and regression. It predicts the output of a data point based on the labels of its nearest neighbors in the feature space. For a given data point x , the distance to all training points x_i is computed, typically using Euclidean distance:

$$d(x, x_i) = \sqrt{\sum_{j=1}^n (x_j - x_{i,j})^2}$$

The k closest neighbors are selected, and the predicted class $\hat{y}$ is determined using majority voting:

$$\hat{y} = \arg \max_c \sum_{i=1}^k \mathbf{1}(y_i = c)$$

where $\mathbf{1}(y_i = c)$ is an indicator function that equals 1 if the neighbor belongs to class c , and 0 otherwise. This method is based on the idea that nearby data points are likely to belong to the same class, allowing KNN to capture local patterns in the data. We find that KNN Accuracy is 0.8345338805595108

2) *Logistic Regression*: In this section, Logistic Regression is applied as a supervised learning model to predict whether a hotel booking will be canceled. The model is trained using the training dataset X_{train} and corresponding labels y_{train} .

Logistic Regression models the probability of the target variable using the sigmoid function:

$$P(y = 1 | x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

where w represents the weight vector and b is the bias term, and y is the actual output set/true class. Since both w and b are learned from the training, it is a parameter and not a hyperparameter. The model learns these parameters by minimizing the log-loss function:

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

The cross terms appear due to the Bernoulli distribution – our binary classification is basically a two level system. After training, predictions are made on the test dataset X_{test} . The predicted class $\hat{y}$ is determined using a threshold (typically 0.5):

$$\hat{y} = \begin{cases} 1, & \text{if } P(y = 1 | x) \geq 0.5 \\ 0, & \text{otherwise} \end{cases}$$

Finally, the performance of the model is evaluated using accuracy, defined as:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}.$$

Also,

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FN} + \text{FP}}$$

This approach allows the model to estimate the probability of booking cancellation and make predictions on unseen data. For our dataset, the accuracy is 0.7932825194739928.

3) *Neural Network (NN)*: In machine learning, a neural network is a model inspired by the structure and function of biological neural networks in animal brains. A key motivation for using multiple layers is that a single layer can only model limited (often linear) relationships, whereas real-world data is highly non-linear and structured. By stacking layers, the network learns hierarchical representations. So, lower layers capture simple features, while deeper layers combine them into more complex patterns, allowing it to approximate complex functions and improve predictive performance.

Forward propagation is the process by which a neural network takes an input and passes it through each layer to produce an output (prediction). At each layer, the input is linearly transformed using weights and biases, and then passed through a non-linear activation function. The forward propagation of a single layer can be mathematically expressed as follows:

Let f be the activation function. Then, for the l^{th} layer, the input is $A^{[l-1]}$ and the output $A^{[l]}$ is computed as:

$$Z^{[l]} = W^{[l]} A^{[l-1]} + B^{[l]} \\ A^{[l]} = f(Z^{[l]})$$

where $W^{[l]}$ is the weight matrix connecting $(l-1)^{\text{th}}$ layer to l^{th} layer, $B^{[l]}$ is the bias matrix of l^{th} layer, and $Z^{[l]}$ is the pre-activation output l^{th} layer.

Backpropagation is used to train a neural network by efficiently computing how each weight and bias contributes to the prediction error, so they can be updated to reduce that error. Mathematical foundation of backpropagation is as follows: let Y be the output, f be the activation function, and E be the cost function. The upstream gradient for the output layer $D_u^{[L]}$, where L is the number of layers in the network, is computed as:

$$D_u^{[L]} = \nabla_Y E$$

Let $D^{[l]} = \nabla_{Z^{[l]}} E$, where, $Z^{[l]}$ be the pre-activation output of l^{th} layer. For l^{th} layer, $D^{[l]}$ is computed as:

$$D^{[l]} = D_u^{[l]} \odot f'(Z^{[l]})$$

Then, the partial derivatives of the cost function with respect to weight matrix $W^{[l]}$ and bias matrix $B^{[l]}$ are computed as:

$$\nabla_{W^{[l]}} E = D^{[l]} A^{[l-1]T}$$

$$\nabla_{B^{[l]}} E = D^{[l]}$$

where $A^{[l-1]}$ is the input from $(l-1)^{th}$ layer. Finally, the weight matrix $W^{[l]}$ and bias matrix $B^{[l]}$ are updated using the following equations:

$$W^{[l]} \rightarrow W^{[l]} - \eta \nabla_{W^{[l]}} E$$

$$B^{[l]} \rightarrow B^{[l]} - \eta \nabla_{B^{[l]}} E$$

where η is the learning rate of the network. In the end, for $(l-1)^{th}$ layer, the upstream gradient $D_u^{[l-1]}$ is computed as:

$$D_u^{[l-1]} = D^{[l]} W_1^{[l]T}$$

We find that for our dataset, the accuracy for neural network is: 0.8602898065164587

4) *Naive Bayes*: In this section, the Gaussian Naive Bayes classifier is applied as a supervised learning model to predict whether a hotel booking will be canceled. The model is trained using the training dataset X_{train} and corresponding labels y_{train} . Naive Bayes is based on Bayes' theorem:

$$P(y | x) = \frac{P(x | y) P(y)}{P(x)}$$

Under the assumption that features are conditionally independent given the class label, the likelihood can be written as:

$$P(x | y) = \prod_{j=1}^n P(x_j | y)$$

In Gaussian Naive Bayes, each feature is assumed to follow a normal distribution:

$$P(x_j | y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_j - \mu_y)^2}{2\sigma_y^2}\right)$$

The predicted class is obtained by maximizing the posterior probability:

$$\hat{y} = \arg \max_y P(y) \prod_{j=1}^n P(x_j | y)$$

The model is evaluated using accuracy on the test dataset, which measures the proportion of correctly predicted instances. The accuracy for Naive Bayes for our dataset is 0.47177318033336124.

VI. RESULT ANALYSIS

In this section, we evaluate the quality of the training using different metrics. Primarily, we will focus on four major evaluation metrics.

A. Accuracy

This calculates the ratio between correct prediction and total prediction.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FN} + \text{FP}}$$

B. Precision

This metric calculates the number of occasion when the prediction was right of all the times the model predicted the cancellation of booking.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

C. AUC (Area Under the ROC Curve)

The Area Under the ROC Curve (AUC) measures the model's ability to distinguish between positive and negative classes across all classification thresholds. It summarizes the performance of the Receiver Operating Characteristic (ROC) curve, which plots the True Positive Rate (TPR) against the False Positive Rate (FPR).

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

An AUC value ranges from 0 to 1, where a value closer to 1 indicates better model performance, and a value of 0.5 corresponds to random guessing. A higher AUC implies that the model is more capable of correctly ranking positive instances higher than negative ones, regardless of the chosen threshold. Intuitively, we have two distribution for positive and negative class. We are interested to see how much they overlap. If they overlap a lot, the model cannot distinguish them. Therefore AUC is low. Otherwise, we find a higher value for AUC.

D. Accuracy vs Precision

Accuracy measures the overall correctness of a model by calculating the proportion of all predictions that are correct. It considers both positive and negative predictions:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Precision, on the other hand, focuses only on the quality of positive predictions. It measures how many of the predicted positive cases are actually correct:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

While accuracy gives a general sense of performance, it can be misleading in imbalanced datasets where one class dominates. Precision is important when the number of false positive predictions is high, because it tells us how many predicted positive cases are actually correct.

E. Precision vs Recall

Precision measures the correctness of positive predictions, i.e., how many of the predicted positive cases are actually positive:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall measures the ability of the model to identify all actual positive cases:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Precision is important when the cost of false positives is high, while recall is crucial when missing a positive case is costly. In practice, there is often a trade-off between the two, and improving one may reduce the other.

Model	Accuracy	Precision	Recall	AUC
KNN	0.8345	0.7892	0.7549	0.9001
Logistic Regression	0.7933	0.8044	0.5839	0.8637
Naive Bayes	0.4718	0.4105	0.9766	0.8013
Neural Network	0.8603	0.8050	0.7785	0.9360

TABLE I

PERFORMANCE COMPARISON OF DIFFERENT CLASSIFICATION MODELS

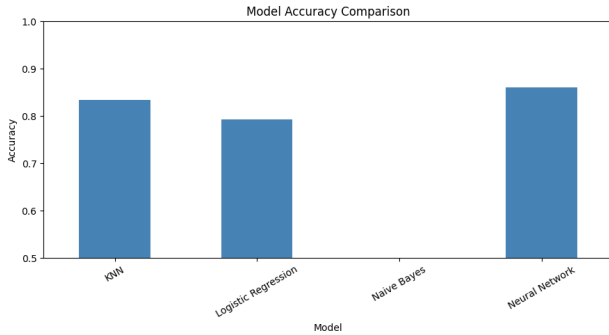


Fig. 13. Accuracy for different supervised machine learning algorithm.

Naive Bayes achieves the highest recall score among all models, indicating that it is the most effective at correctly identifying actual cancellation cases. On the other hand, the neural network achieves the highest accuracy and AUC among all the models, indicating that it not only makes the most correct predictions overall but also has the best ability to distinguish between cancellation and non-cancellation cases.

F. Confusion Matrix Analysis

A confusion matrix is a tabular representation used to evaluate the performance of a classification model by comparing actual and predicted labels. For binary classification, it consists of four components: true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). This representation provides deeper insight into model behavior beyond overall accuracy by explicitly showing the types of errors made.

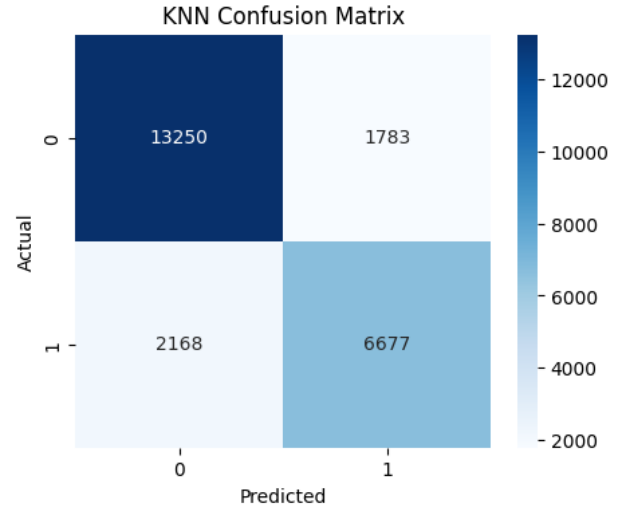


Fig. 14. confusion matrix for KNN.

The confusion matrices provide a clearer picture of how each model behaves on both classes. For the KNN model, there are 13,250 true negatives and 6,677 true positives, indicating that it correctly classifies a large portion of both non-cancellations and cancellations. However, it produces 2,168 false negatives, meaning a noticeable number of actual cancellations are missed, along with 1,783 false positives. This suggests that while KNN is relatively balanced, it still struggles somewhat in detecting all positive cases.

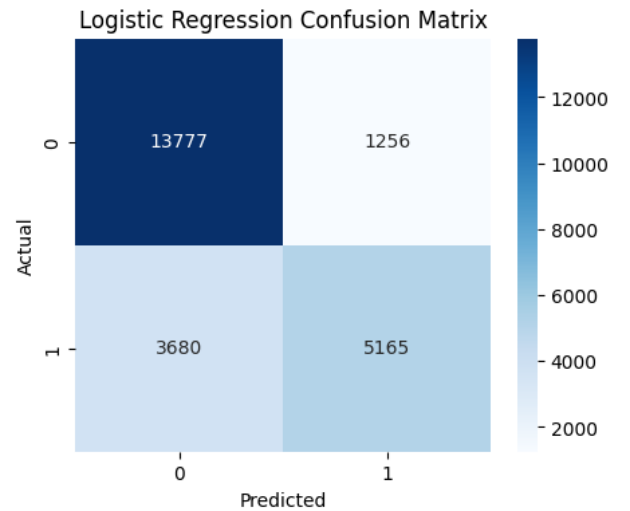


Fig. 15. confusion matrix for logistic regression.

For Logistic Regression, the model achieves 13,777 true negatives and only 1,256 false positives, showing that it is very effective at correctly identifying non-cancellations and avoids raising false alarms. However, it produces 3,680 false negatives and only 5,165 true positives, indicating that a significant number of actual cancellations are not detected. This behavior reflects a conservative model that favors predicting the negative class, leading to lower recall for cancellations.

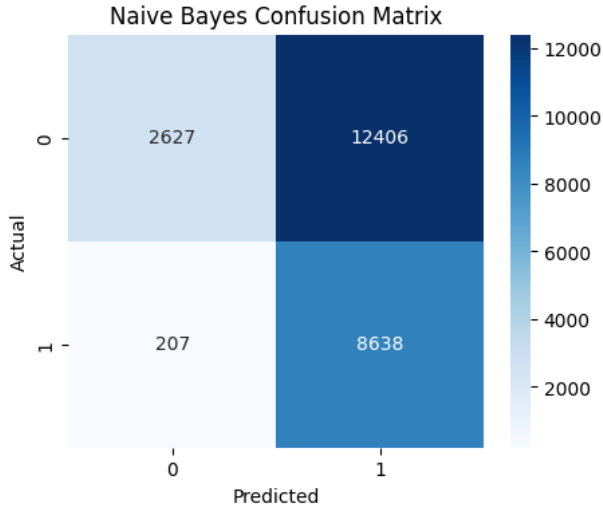


Fig. 16. confusion matrix for Naive Bayes.

In contrast, the Naive Bayes model shows a very different behavior. It correctly identifies 8,638 true positives and has only 207 false negatives, indicating that it successfully detects almost all actual cancellations and thus achieves very high recall. However, this comes at a significant cost: the model produces 12,406 false positives while only identifying 2,627 true negatives. This means that a large number of non-cancellations are incorrectly predicted as cancellations, suggesting that the model is heavily biased toward predicting the positive class.

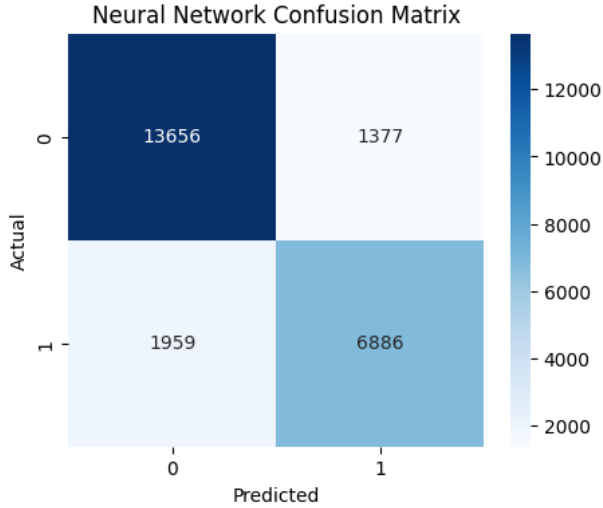


Fig. 17. confusion matrix for KNN.

The Neural Network model provides the most balanced performance among all models. It achieves 13,656 true negatives and 6,886 true positives, while keeping false positives (1,377) and false negatives (1,959) relatively low compared to other models. This indicates that the model is effective at correctly identifying both cancellations and non-cancellations without strongly favoring one class over the other, resulting in a better overall trade-off between precision and recall.

Overall, the confusion matrix analysis highlights the trade-offs between different models, with the neural network provid-

ing the best balance between correctly identifying both classes and minimizing classification errors.

G. Receiver Operating Characteristic (ROC) Curve

The ROC curve is a graphical representation of a classification model's performance. It plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at different threshold values.

- **False Positive Rate (FPR):**

$$FPR = \frac{FP}{FP + TN}$$

- **True Positive Rate (TPR):**

$$TPR = \frac{TP}{TP + FN}$$

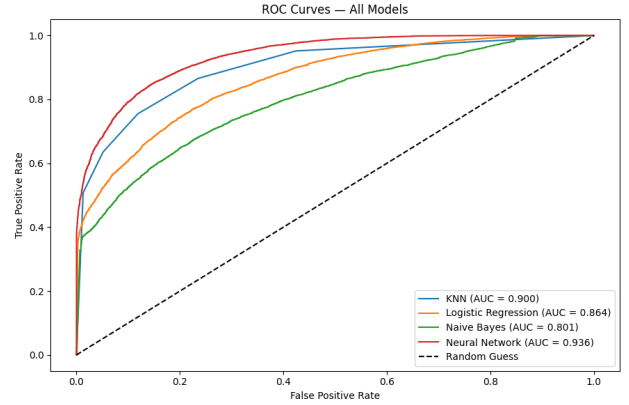


Fig. 18. ROC curve

The ROC curve compares the performance of different classification models by plotting the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold values. The dashed diagonal line represents a random classifier with no predictive power. All models perform better than random guessing, as their curves lie above this line. Among the models, the Neural Network achieves the highest Area Under the Curve (AUC = 0.936), indicating the best ability to distinguish between canceled and non-canceled bookings. KNN also performs well (AUC = 0.900), followed by Logistic Regression (AUC = 0.864), while Naive Bayes shows comparatively lower performance (AUC = 0.801). Overall, the ROC analysis demonstrates that the Neural Network is the most effective model for this classification task.

VII. DISCUSSION AND SUMMARY

In this study, different models (KNN, Logistic Regression, Naive Bayes, and Neural Network) were used to predict booking cancellations. Their performance was compared using metrics such as accuracy, precision, recall, F1-score, AUC, and confusion matrices. The results show that each model behaves differently and makes different types of errors.

KNN gives a fairly balanced result, correctly predicting both cancellations and non-cancellations, but it still misses some actual cancellations. Logistic Regression is good at identifying

non-cancellations and makes fewer false alarms, but it fails to detect many cancellations, leading to lower recall. Naive Bayes performs very well in detecting cancellations, but it incorrectly predicts many non-cancellations as cancellations, resulting in a high number of false positives.

The Neural Network performs the best overall. It correctly identifies both cancellations and non-cancellations while keeping errors relatively low. This balance is also reflected in its higher accuracy, F1-score, and AUC compared to the other models.

In conclusion, the choice of model depends on the goal of the task. If it is more important to detect all cancellations, a model like Naive Bayes may be useful. However, if a balanced and reliable performance is required, the Neural Network is the most suitable option in this case.